

클로드 하네스 활용을 통한 프로젝트 관리 최적화

1. 목적

공공기관 SI 프로젝트 수행 과정에서 반복적으로 발생하는 문서 품질 저하, 요구사항 추적성 단절, 공정률 왜곡, 산출물 취합 지연, 개발 품질 저하 문제를 개선하기 위해, Jira·Confluence·Slack·Claude 등 AI 기반 협업 도구를 활용한 프로젝트 관리 및 개발 최적화 방안을 체계를 수립하는데 목적이 있다.

특히 공공기관 프로젝트는 전체 공정의 상당 부분이 사업관리 및 개발 산출물 작성·검토·취합에 집중되어 있으며, 감리·검수 단계에서 산출물 품질과 요구사항 추적성이 주요 점검 대상이 된다. 이에 따라 AI를 단순 코딩 보조 도구로 활용하는 수준을 넘어, 프로젝트 전 공정의 역할·룰·스킬을 표준화하고 자동화하는 업무 혁신 체계를 수립하고자 한다.

2. 추진 배경

공공기관 프로젝트는 전통적으로 폭포수 모델 중심으로 운영되어 왔으며, 요구사항 정의, 분석, 설계, 개발, 테스트, 감리, 검수 단계가 순차적으로 진행된다. 그러나 실제 수행 과정에서는 다음과 같은 문제가 반복적으로 발생한다.

첫째, 산출물 작성과 취합이 수작업 중심으로 이루어져 문서 품질 확보에 많은 시간이 소요된다.

둘째, 요구사항 추적관리가 최초 작성 시점에는 이루어지지만, 개발과 테스트가 진행되면서 지속적으로 유지되지 않는다.

셋째, 주간보고와 월간보고 작성 시 WBS, 이슈, 산출물, 결함, 개발 진행률을 별도 담당자가 수작업으로 취합하고 있다.

넷째, 요구사항이 불명확하거나 테스트 기준이 부족한 상태에서 개발이 진행되면 오픈 시점에 다수의 오류가 발생하고, 이후 긴급 수정과 재작업 비용이 증가한다.

다섯째, 공공기관 특성상 폭포수 방식이 강하게 적용되어 개발 완료 후 품질 문제가 집중적으로 드러나는 구조가 발생한다.

3. 추진 방향

본 프로젝트 관리 최적화의 핵심 방향은 다음과 같다.

문서관리 자동화 → 요구사항 추적관리 자동화 → 공정관리 자동화 → SDD + TDD 기반 요구사항작성 → AI 기반 역할·룰·스킬 정착 → 전자정부표준프레임워크 최적화 개발 → 배포 → 통합테스트 → 산출물작성

이를 위해 Jira, Confluence, Slack, Claude를 기본 협업 도구로 활용하고, 필요 시 Spec Kit, Claude Code, 기타 AI CLI 도구를 도입하여 프로젝트 관리와 개발 수행 전반에 AI 자동화 체계를 적용한다.

4. 주요 개선 목표

4.1 문서 품질 확보

정부기관 프로젝트는 사업관리 및 개발 산출물 관리 비중이 높으며, 산출물 품질은 감리·검수 대응의 핵심 요소이다.

구분	기존 방식	개선 방식
문서 작성	담당자별 개별 작성	Confluence 표준 템플릿 기반 작성
문서 취합	PM/문서 담당자 수작업 취합	Jira/Confluence 기반 자동 취합
문서 검토	개별 검토 의견 관리	Claude 기반 누락/불일치 검토
버전 관리	파일명 중심 관리	Confluence Page + Attachment + 기준선 관리
보고 자료	수작업 복사/정리	자동 보고서 초안 생성

기대 효과는 산출물 취합 및 검토 시간을 단축하고, 감리·검수 단계에서 문서 품질 관련 리스크를 사전에 줄이는 것이다.

4.2 요구사항 추적관리 강화

공공기관 프로젝트에서는 요구사항 ID를 기준으로 유스케이스, 화면, API, 프로그램, 테이블, 테스트케이스까지 추적해야 한다. 단계별 산출물 간 ID 연결이 유지되지 않아 감리 시 많은 시간을 소요한다.

개선 방향은 다음과 같다.

요구사항추적표를 기준으로 jira 필드에 다음사항을 추가하여 집중관리한다.
요구사항 ID → 유스케이스 ID → 화면 ID → API ID → 프로그램 ID → 테이블 ID → 테스트케이스 ID → 결함 ID → 검수 승인 ID

AI를 활용하여 기존 문서의 요구사항 ID 연결 상태를 자동 점검하고, 누락되거나 불일치한 항목을 식별하여 Jira와 Confluence 문서에 자동 업데이트한다.

관리 항목	개선 내용
요구사항 누락	Claude가 요구사항정의서와 유스케이스정의서 비교
ID 불일치	문서 간 요구사항 ID, 테스트케이스 ID 연결 점검
변경 반영	변경된 요구사항을 관련 설계·테스트 문서에 반영
감리 대응	요구사항 추적표 자동 생성 및 최신화
보완 요청	Jira Sub-task 또는 댓글로 담당자에게 보완 요청

이를 통해 감리 지적 가능성을 줄이고, 요구사항 변경에 따른 설계·개발·테스트 영향도를 체계적으로 관리할 수 있다.

4.3 공정관리 자동화

현재 주간보고, 월간보고 작성 시 WBS, Jira 이슈, 산출물 현황, 개발 진행률, 결함 현황을 별도 담당자가 수작업으로 취합하고 있다. 이 과정은 반복적이고 시간이 많이 소요되며, 개발자의 주관적 보고에 따라 공정률이 왜곡될 수 있다.

개선 방향은 기존의 “개발자 보고 기반 공정률”을 “TDD 통과 기준 기반 공정률”로 전환하는 것이다.

기존 공정률 기준	개선 공정률 기준
개발자 주관적 진행률 입력	Jira Task 완료 상태
PM 수작업 취합	Jira 자동 집계
산출물 별도 확인	Confluence Page Properties 기반 취합
테스트 결과 수동 확인	TDD/테스트 통과 여부 기반 확인
보고서 수작업 작성	AI 기반 주간·월간보고 자동 생성

공정률 산정 기준 예시는 다음과 같다.

요구사항 명세 완료: 10% > 설계 문서 승인: 20% > 테스트케이스 작성 완료: 30% > TDD 테스트 작성 완료: 40% > 단위 테스트 통과: 60% > 통합 테스트 통과: 80% > 고객 검토 및 승인: 100%

이를 통해 허위 보고 가능성을 줄이고, 실제 산출물과 테스트 결과에 기반한 객관적 공정관리가 가능해진다.

4.4 개발 자동화 및 품질 향상

정확한 요구사항 명세를 입력으로 사용하여 Claude Code, Spec Kit, AI CLI 도구를 통해 기능 구현과 테스트를 자동화한다. 개발 자동화의 핵심은 단순 코드 생성이 아니라, 다음 흐름을 정착시키는 것이다.

요구사항정의서 → 유스케이스정의서 → 화면정의서 → 테이블정의서 → Spec 작성 → Plan 작성 → TDD 작성 → Task 분해 → 코드 구현 → 테스트 통과 → 산출물 업데이트

이를 통해 개발자는 반복적인 보일러플레이트 코드 작성보다 요구사항 검증, 예외 처리, 품질 개선에 집중할 수 있다.

4.5 애자일 프로젝트 정착

공공기관 프로젝트는 폭포수 모델 기반으로 수행되는 경우가 많아 개발 후반부 또는 오픈 시점에 다수의 오류가 집중적으로 발견된다. 공공기관의 공식 산출물 체계는 유지하되, 내부 개발 프로세스는 애자일 방식으로 전환하여 품질과 대응 속도를 높인다.

기존 방식	개선 방식
전체 설계 후 일괄 개발	요구사항 단위 점진 개발
오픈 시점 오류 집중	프로토타입 기반 조기 검증
문서와 개발 분리	문서·테스트·코드 연계
테스트 후행	TDD 기반 테스트 선행
고객 피드백 지연	반복 검토 및 보완

5. AI 기반 업무 혁신 방향

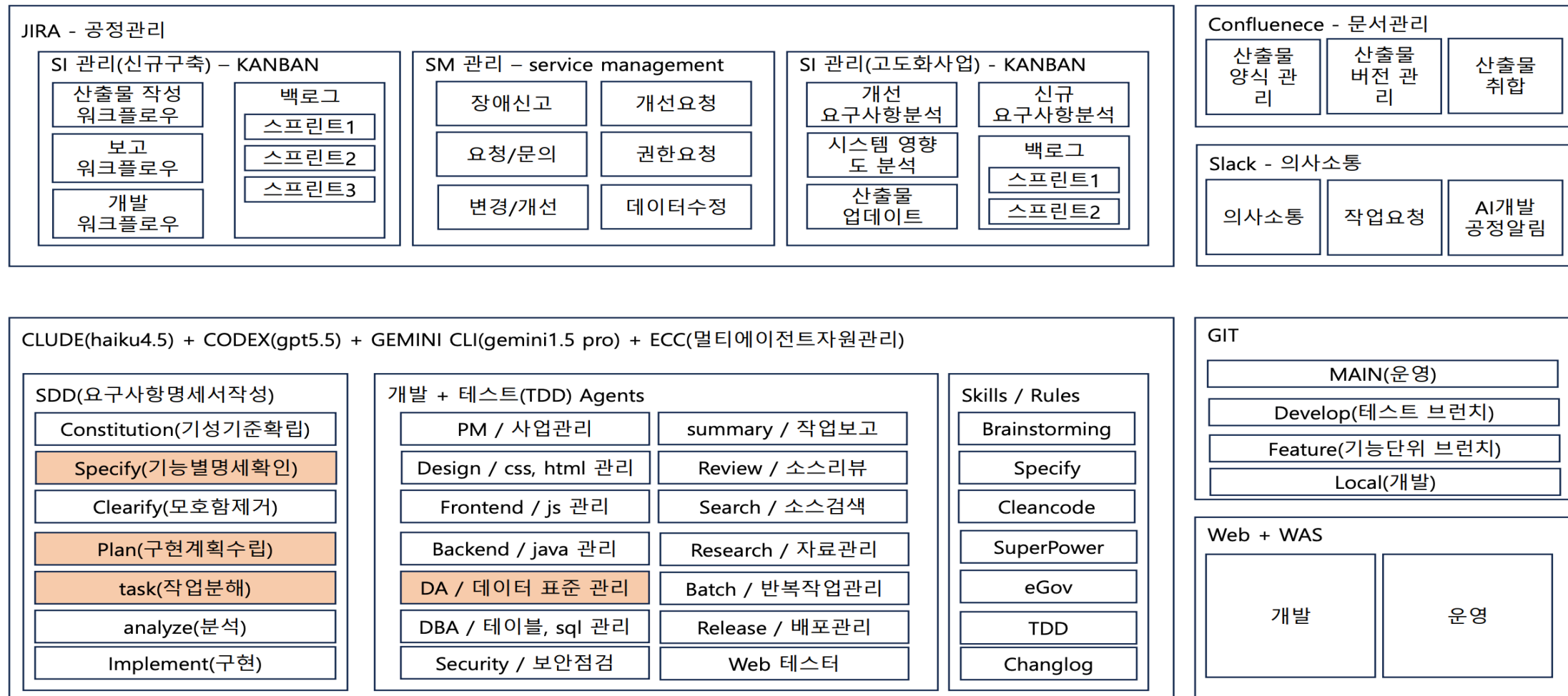
본 과제의 핵심은 AI CLI 도구를 활용해 코딩만 자동화하는 것이 아니다. 프로젝트 공정에서 발생하는 역할별 룰과 워크플로우를 정리하고, AI가 이를 수행하거나 보조할 수 있도록 체계화하는 것이다.

AI 적용 대상은 다음과 같다.

영역	AI 활용 방안
문서관리	산출물 템플릿 생성, 누락 검토, 문서 비교
요구사항관리	요구사항 체계화, ID 추적, 변경 영향도 분석
공정관리	Jira Task 기반 공정룰 산정, 보고서 자동 생성
개발관리	Spec 기반 구현 계획, TDD, 코드 생성
테스트관리	테스트케이스 생성, 테스트 누락 점검
감리대응	요구사항 추적표, 산출물 현황, 보완사항 정리
운영관리	이슈 요약, 장애보고서, FAQ, 조치 이력 지식화

6. 목표 시스템 구성

정부기관 SI / SM 프로젝트 공정관리 및 AI 개발시스템 구성도



7. 핵심 아키텍처

claude-code-orchestrator: 멀티 에이전트로 만드는 개발혁신

ARCHITECTURE OVERVIEW(아키텍처 개요)

Specialist Agents

Specialist Agents

Codex CLI
(Deep Reasoning, Design, Debug)
Codex CLI는 깊은 추론과 디버깅을 분담한다.

Claude Code (Orchestrator)
시스템의 지휘탑으로서, 사용자 대응, 작업(태스크) 열람, 그리고 전체 실행 관리를 담당한다.

Gemini CLI
(1M Context, Research, Multimodal)
Gemini CLI는 리서치와 1M 토큰의 방대한 컨텍스트 기반 분석을 분담한다.

CONTEXT MANAGEMENT (컨텍스트 관리 전략)

200k Context
(관련성 판단)

정보를
효율적으로 유지

Subagent Pattern: Context 절약을 위한 위임

**Main
Orchestrator**
(Claude Code)

Summary
Result
Summary
Result (요약
만 제공)

Subagent
(Independent
Context)
Large Text/Output
(대규모 텍스트 및 포괄적 분석)

Subagent
특정 Task를 독립된
Subagent에 병렬 실행
시킨다.

AGENT CAPABILITIES (각 에이전트의 역할)

Agent	Role/Capabilities	Primary Use Case
Claude	Orchestration, User Interaction	User Interaction, Planning, 전체 조율
Codex	Deep Reasoning, Design, Debug	설계 Trade-off 분석, 복잡한 Bug 수정
Gemini	1M Context, Research, Multimodal	대규모 Code 분석, Document 조사, Video/PDF 처리

WORKFLOW: /startproject (프로젝트 시작 플로우)

Phase1: Gemini (Research)
Repository 현황분석 및 필요한 정보 사전 조사 수행

Phase2: Claude (Planning)
사용자 요구사항 Hearing 후, 전체 Implementation Plan 수립

Phase3: Codex (Review)
작성한 계획을 기술적 관점에서 Review, Risk 분석 수행

Phase4: Claude (Task List)
Review 결과를 반영해 최종 Execution Task List 생성

DIRECTORY STRUCTURE (디렉토리 구성)

.claude/ (System Core)

- skills/ (워크플로우)
- rules/ (가이드라인)
- hooks/ (자동화 스크립트)
- docs/ (설계 문서)
- logs (로그)

.codex/ (Codex CLI Config)

- config.toml (설정 파일)

.gemini/ (Gemini CLI Config)

- config.toml (설정 파일)

CLAUDE.md (Main System Doc)

프로젝트 전체의 Main System Document 역할

KEY SKILLS & HOOKS (주요 스킬과 자동화)

/tdd (Test Driven Development)

- Test Design, Red(Fail), Green(Success), Refactor 사이클 자동화

agent-router.py (Hook)

- 사용자 입력을 분석해 최적의 Agent(Codex/Gemini)로 Routing 제안

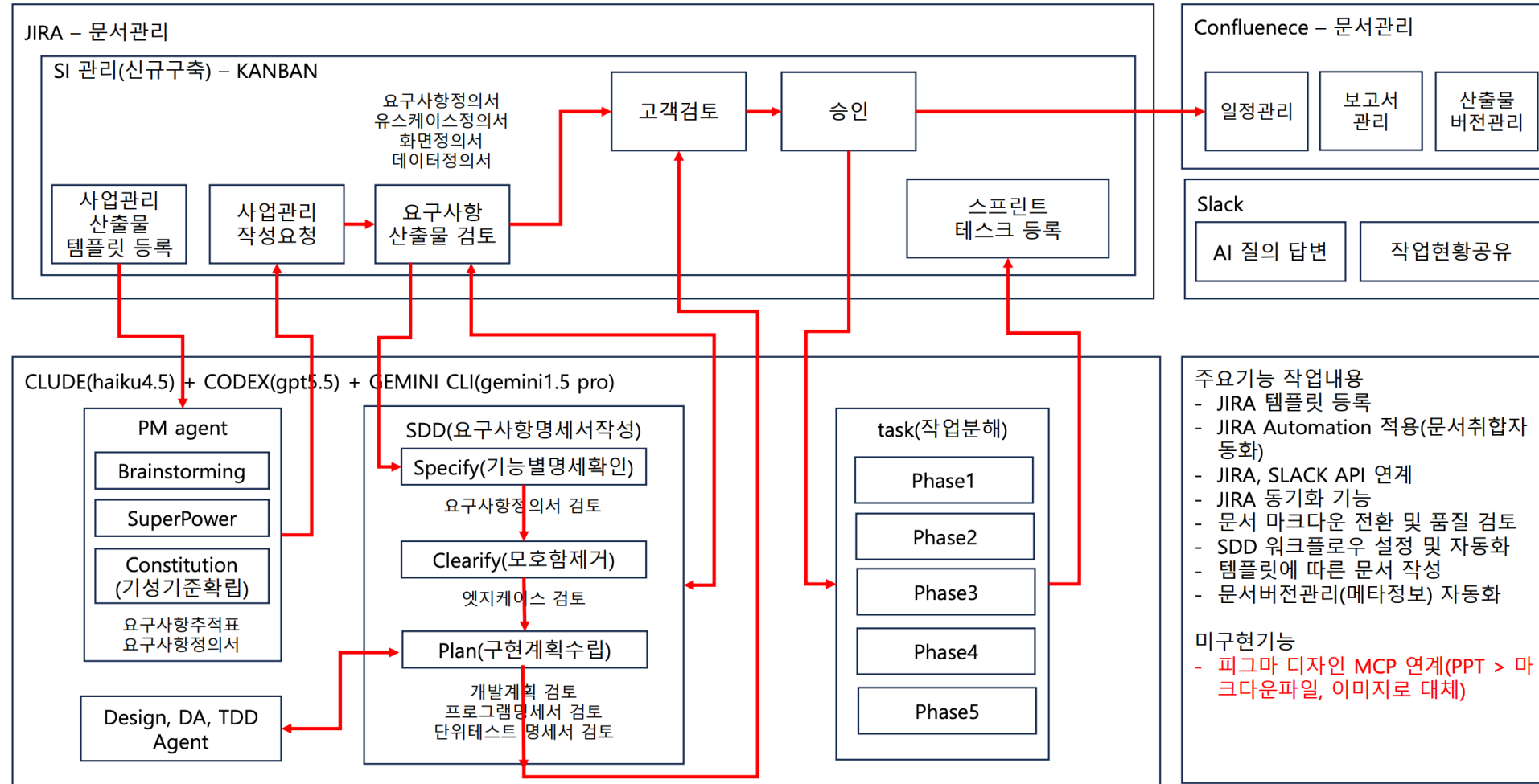
/checkpointing

- Session History 및 File Change 저장, 작업 연속성 보장

MD

7.1 사업관리

정부기관 SI / SM 프로젝트 공정관리 및 AI 개발시스템 **SDD-WORKFLOW**(요구사항명세서 작성)

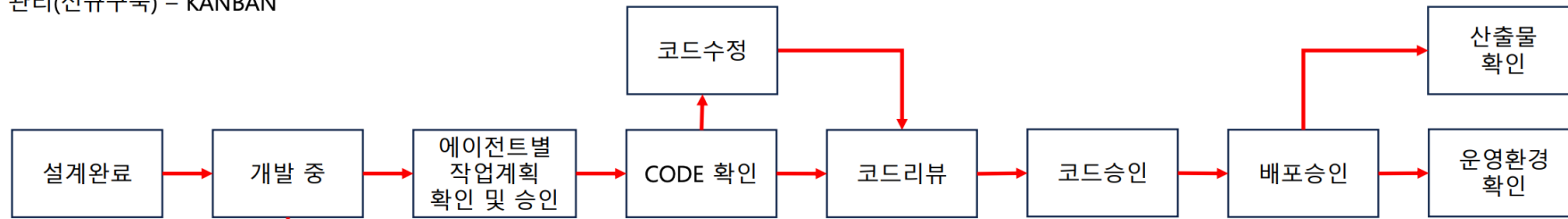


7.2 개발공정관리

정부기관 SI / SM 프로젝트 공정관리 및 AI 개발시스템 **Agent workflow(개발공정관리)**

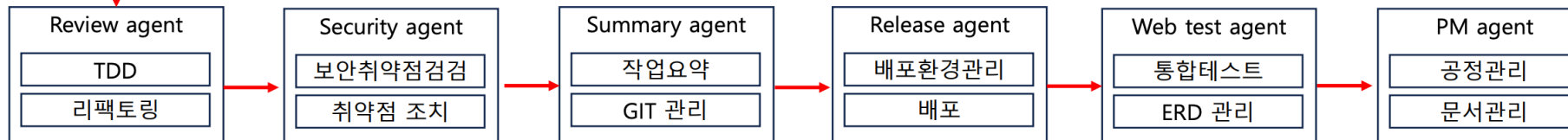
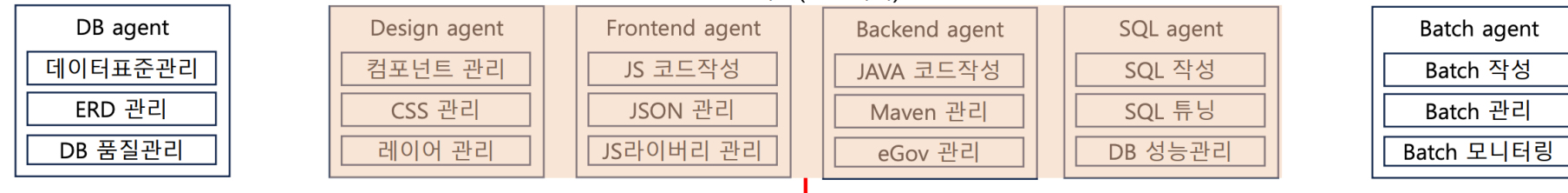
JIRA - 스프린트 관리

SI 관리(신규구축) - KANBAN



CLUDE(haiku4.5) + CODEX(gpt5.5) + GEMINI CLI(gemini1.5 pro)

코드작성(병렬처리)



7.3 에이전트 구성도

8. 담당자별 역할

담당	역할	주요 내용
신대원	AI 툴 관리	Jira, Confluence, Slack, Claude 도입 및 AI 도구 관리
	에이전트·룰·스킬 관리	AI 설정, 프롬프트 규칙, 에이전트 룰, 스킬 관리
	전자정부프레임워크 테스트	전자정부프레임워크 기반 코딩 자동화 테스트 수행
	추진 총괄	AI 기반 프로젝트 관리 최적화 방향 수립 및 성과 관리
윤지혜	문서관리 전략 수립	정부기관 문서 템플릿 등록 및 산출물 관리 체계 수립
	Jira/Confluence/Slack 문서관리	문서 작성·검토·취합 전략 수립
	공정관리 자동화	Jira Task 취합 및 주간·월간 보고서 자동 생성
	산출물 품질관리	산출물 누락, 검토·승인 상태 관리
강호일	에이전트 최적화	KOMSA 개발환경 맞춤 공통기능, 솔루션, API 지원
	개발환경 표준화	개발환경, 공통 모듈, API 명세 정리
	업무 테스트	해수호봇, RCMS 업무 AI 에이전트 성능 검증
	기술 검증	생성 코드 품질, 테스트 통과율, 생산성 검증
박상원	컴포넌트 최적화	KOMSA 화면 패턴 분석 및 컴포넌트 명세 설정
	화면 패턴 표준화	반복 화면, 공통 UI, 입력/조회/상세/승인 패턴 정리

9. 역할별 R&R 요약

구분	신대원 부장	윤지혜 과장	강호일 과장	박상원 대리
AI 도구 도입/관리	주관	지원	지원	지원
에이전트 룰/스킬 관리	주관	지원	지원	지원
문서 템플릿 관리	검토	주관	지원	지원
산출물 관리 전략	검토	주관	지원	지원
공정관리 자동화	검토	주관	지원	지원
개발환경 최적화	검토	지원	주관	지원
공통기능/API 정리	검토	지원	주관	지원
컴포넌트 명세 최적화	검토	지원	지원	주관
업무 테스트	총괄	지원	해수호봇, RCMS	운항관리, 항만보안심사
성과 측정	주관	지원	지원	지원

10. 기대 효과

10.1 정량 효과

구분	기대 효과
문서 취합 시간	Confluence/Jira 자동 취합으로 단축
문서 검토 시간	Claude 기반 사전 검토로 단축
보고서 작성 시간	주간·월간보고 자동 생성으로 단축
요구사항 추적 누락	AI 기반 추적표 점검으로 감소
공정률 왜곡	TDD 통과 기준 기반 산정으로 감소
오픈 후 오류	프로토타입·TDD 기반 개발로 감소

10.2 정성 효과

구분	기대 효과
프로젝트 투명성	Jira 기반 실시간 공정 확인
감리 대응력	요구사항 추적성과 산출물 품질 향상
개발 품질	명확한 요구사항과 테스트 기준 기반 구현
협업 효율	Slack/Jira/Confluence 연계로 소통 비용 감소
업무 표준화	역할별 룰과 워크플로우 정착
AI 활용 수준 향상	단순 코딩 보조에서 프로젝트 관리 자동화로 확장

11. 단계별 추진 계획

11.1 1단계: 기반 구축

항목	내용
도구 구성	Jira, Confluence, Slack, Claude 기본 환경 구성
문서 템플릿	정부기관 산출물 템플릿 등록
Jira 구조	프로젝트, Issue Type, Workflow, Custom Field 설계
Confluence 구조	산출물 Page Tree 및 AI Context 영역 구성
Slack 채널	문서 요청, 검토, 보고, 개발, 이슈 채널 구성

11.2 2단계: 문서·요구사항 자동화

항목	내용
문서 취합	Confluence Page Properties 기반 산출물 현황 취합
요구사항 체계화	Claude/Speckit 기반 requirements.md, spec.md 생성
추적성 점검	요구사항-설계-테스트 ID 연결 자동 점검
보완 요청	Jira 댓글 또는 Task로 보완사항 등록

11.3 3단계: 공정·보고 자동화

항목	내용
WBS 자동화	Jira Task 기반 WBS 자동 생성
공정률 산정	TDD 통과 기준 기반 진척률 계산
주간보고	Jira/Confluence/Slack 데이터 취합 후 자동 생성
월간보고	월간 공정, 이슈, 리스크, 산출물 현황 자동 보고

11.4 4단계: 개발 자동화 및 검증

항목	내용
개발 자동화	Claude Code, Spec Kit 기반 코드 생성 테스트
TDD 적용	테스트 우선 개발 기준 수립
컴포넌트 최적화	KOMSA 화면 패턴 기반 컴포넌트 명세화
업무 검증	해수호봇, RCMS, 운항관리, 항만보안심사 적용 검증

12. 성과 측정 지표

지표	측정 방법
문서 취합 소요시간	기존 수작업 대비 자동 취합 시간 비교
문서 검토 소요시간	검토 요청부터 승인까지 평균 시간
요구사항 추적 누락 건수	감리/내부 점검 시 누락 건수
주간보고 작성 시간	보고서 작성 담당자 투입 시간
공정률 정확도	Jira/TDD 기준 공정률과 실제 완료율 비교
테스트 통과율	요구사항별 테스트 통과 비율
AI 생성 코드 활용률	AI 생성 코드 중 반영된 코드 비율
오픈 후 결함 건수	기존 프로젝트 대비 초기 결함 건수 비교

13. 리스크 및 대응 방안

리스크	내용	대응 방안
AI 결과 신뢰성	AI가 잘못된 분석 또는 코드를 생성할 수 있음	담당자 검토와 승인 절차 유지
문서 품질 편차	입력 문서가 부실하면 AI 결과도 부정확	표준 템플릿과 필수 항목 강제
팀원 적응 부담	Jira/Confluence/AI 도구 사용 부담	역할별 교육 및 단계적 적용
보안 이슈	프로젝트 문서와 개인정보 외부 전송 우려	사내 보안 정책 및 입력 데이터 마스킹 적용
과도한 자동화	검토 없이 자동 반영 시 오류 확산	승인 기반 워크플로우 적용
도구 종속성	특정 AI 도구에 의존 가능성	Claude 외 대체 도구 검토 및 표준 인터페이스 유지

14. 임원 의사결정 요청사항

본 과제 추진을 위해 다음 사항에 대한 의사결정이 필요하다.

요청사항	내용
추진 방향 승인	AI 기반 프로젝트 관리 최적화 과제 추진 승인
도구 사용 승인	Jira, Confluence, Slack, Claude 및 필요 AI 도구 사용 승인
역할 배정 승인	담당자별 R&R 확정
시범 적용 승인	해수호봇, RCMS, 운항관리, 항만보안심사 대상 시범 적용
보안 검토 승인	AI 도구 사용 시 문서·데이터 보안 기준 수립
성과 측정 승인	문서시간, 공정률, 테스트통과율 등 KPI 기반 성과 측정

15. 결론

AI 기반 프로젝트 관리 최적화는 단순히 개발자가 AI로 코드를 생성하는 수준의 도입이 아니다. 공공기관 프로젝트의 핵심 리스크인 문서 품질, 요구사항 추적성, 공정률 정확성, 개발 품질, 감리 대응력을 개선하기 위한 프로젝트 수행 방식의 전환이다.

Jira, Confluence, Slack, Claude를 중심으로 프로젝트 역할과 워크플로우를 표준화하고, Spec Kit 및 AI CLI 도구를 활용하여 요구사항 분석, 계획 수립, TDD, 작업 분해, 보고서 생성까지 자동화하면 다음 효과를 기대할 수 있다.

문서 품질 향상
요구사항 추적성 강화
공정관리 정확도 향상
보고 업무 시간 단축
개발 품질 향상
감리 및 검수 대응력 강화
AI 기반 업무혁신 체계 정착

따라서 본 과제는 단기적으로는 프로젝트 관리 업무의 효율화를 목표로 하며, 중장기적으로는 공공 SI 프로젝트 수행 체계를 AI 기반으로 고도화하는 전략 과제로 추진할 필요가 있다.